Primitives vs References

11 min read Updated 28 Jul 2026 Topic 3 of 65

Bookmark Mark done

Why it matters

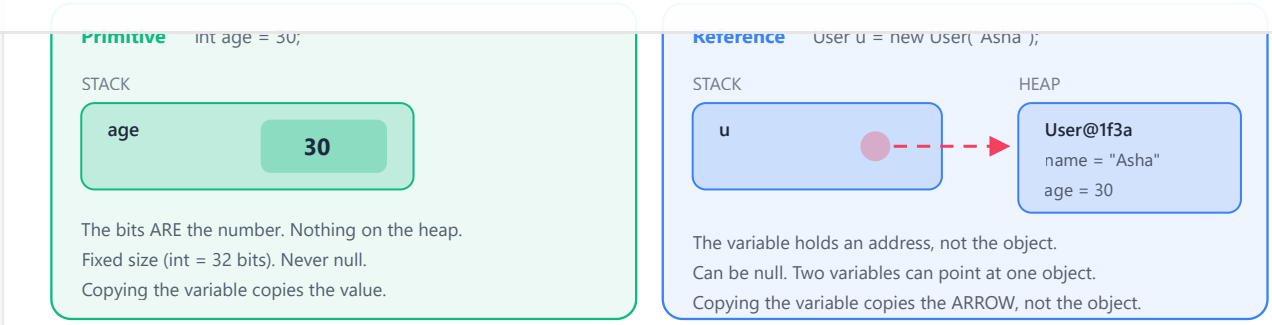
Java has exactly two kinds of value, and almost every confusing thing a beginner hits comes from not knowing which one they are holding. Why does `==` work on `int` but betray you on `String`? Why can a variable be `null` here but not there? Why does modifying an object inside a method change the caller's object, while assigning to it does not? All of it — including the whole of the next two topics — is downstream of this single distinction.

Interviewers use it as a depth probe precisely because it is easy to answer shallowly. "Primitives store values, objects store references" is table stakes. What they are listening for is whether you can predict behaviour you have not memorised.

The concept, simply

A **primitive** variable holds the actual value, in its own slot. There are exactly eight primitive types, they are built into the language, and they are not objects.

A **reference** variable holds a handle that lets the JVM find an object elsewhere in memory. Everything that is not one of the eight primitives — every class, every array, every interface, every enum, every `String` — is a reference type.



The whole distinction in one picture: a primitive slot contains the value; a reference slot contains a way to reach the value.

The eight primitives, with the numbers interviewers actually ask for:

Type	Size	Range / values	Default	Wr
`byte`	8 bits	-128 … 127	`0`	By
`short`	16 bits	-32,768 … 32,767	`0`	Sh
`int`	32 bits	≈ -2.1 bn … 2.1 bn	`0`	In
`long`	64 bits	≈ -9.2×10^{18} … 9.2×10^{18}	`0L`	Lo
`float`	32 bits	IEEE 754 single precision	`0.0f`	Fl
`double`	64 bits	IEEE 754 double precision	`0.0d`	Do
`char`	16 bits	`0` … `65,535` (unsigned UTF-16 unit)	`'\u0000'`	Ch
`boolean`	not specified	`true` / `false`	`false`	Bo

Two of those rows are interview bait. `char` **is unsigned** — it is the only unsigned primitive in Java, holding a UTF-16 code unit. And `boolean` **'s size is deliberately unspecified** by the JVM specification; in practice HotSpot uses a full byte in arrays and typically an int-sized slot elsewhere. Anyone who confidently says "a boolean is 1 bit" has learned it from a bad source.

How it works

The behavioural consequences are what actually matter, and they all follow mechanically from the picture above.

Nullability. A reference can point at nothing — that is `null`. A primitive slot always contains a valid bit pattern, so it can never be `null`. `int x = null;` does not compile.

Equality. `==` compares the contents of the two slots. For primitives that is the values, which is exactly what you want. For references it compares the *addresses* — so it asks "are these the same object?", not "do these objects mean the same thing?". That second question is what `equals()` is for, and it is the source of the single most common Java bug (Chapter 2 and Chapter 3 return to it repeatedly).

Default values. *Fields* are initialised automatically — numeric primitives to zero, `boolean` to `false`, references to `null`. *Local variables* are not: the compiler requires definite assignment before use, so reading an uninitialised local is a compile error, not a garbage value. That difference trips people up constantly.

Memory. A primitive local lives directly in the stack frame. A reference local lives in the stack frame, but the object it points at lives on the heap. A primitive *field* lives inside its object on the heap, alongside the object's other fields.

Generics. Java generics only work over reference types. `List<int>` does not compile; you need `List<Integer>`. This is a direct consequence of type erasure (Chapter 5) and is the main reason the wrapper classes exist at all.

Code examples

Assignment behaves completely differently for each kind, and this is the whole topic in eight lines:

JAVA

Copy

```java
// Primitive: the VALUE is copied. Two independent slots.
int a = 10;
int b = a;      // b gets its own copy of 10
b = 20;
```

```
System.out.println(a);   // 10  - a never moved
// Reference: the ARROW is copied. One object, two names.
int[] x = {1, 2, 3};
int[] y = x;    // y gets a copy of the reference - NOT a copy of the arra
y[0] = 99;
System.out.println(x[0]);  // 99  - same array, seen through two variables

y = new int[]{7, 8, 9};    // reassigning y just re-points y
System.out.println(x[0]);  // 99  - x still points at the original
```

Equality, and the classic trap:

JAVA Copy

```
int p = 127, q = 127;
System.out.println(p == q);            // true - comparing numbers

String s1 = new String("hi");
String s2 = new String("hi");
System.out.println(s1 == s2);          // false - two distinct objects
System.out.println(s1.equals(s2));     // true  - same characters

// And the one that gets everyone (Chapter 3 explains the cache):
Integer i1 = 127, i2 = 127;
Integer i3 = 128, i4 = 128;
System.out.println(i1 == i2);          // true  - both from the Integer ca
System.out.println(i3 == i4);          // false - 128 is outside the cache
System.out.println(i3.equals(i4));     // true  - always use equals()
```

Fields versus locals:

JAVA Copy

```
public class Defaults {
    int count;          // field   → 0
    boolean active;     // field   → false
    String name;        // field   → null

    void demo() {
```

```
        int local;
        // System.out.println(local);  // ✗ does not compile:
        //                             "variable local might not have b
        local = 5;
        System.out.println(local);      // ✓ 5
    }
}
```

And the arithmetic edge that comes from fixed sizes:

JAVA Copy

```java
int max = Integer.MAX_VALUE;       // 2_147_483_647
System.out.println(max + 1);       // -2147483648  — silently wraps around

// Java 8+ gives you the loud version when wrapping would be a bug:
try {
    Math.addExact(max, 1);
} catch (ArithmeticException e) {
    System.out.println(e.getMessage());   // "integer overflow"
}

// The classic long-overflow bug: the RIGHT-hand side is computed as int
// arithmetic and overflows BEFORE the widening to long ever happens.
long wrong = 1000 * 60 * 60 * 24 * 30;    // -1702967296  ✗
long right = 1000L * 60 * 60 * 24 * 30;   // 2592000000   ✓ one L fixes it
```

Under the hood

Why keep primitives at all? Purity would say make everything an object, as Smalltalk and Ruby do. Java kept primitives for performance, and the gap is not small. An `int[]` of a million elements is a single contiguous four-megabyte block. An `Integer[]` of a million elements is a million pointers *plus* a million separate heap objects, each carrying a 12–16 byte header — roughly four to five times the memory, spread across the heap so that every access risks a cache miss, and every element is extra work for the garbage collector. For numeric work the difference is order-of-magnitude, not marginal.

JVMs with heaps under about 32 GB it applies **compressed oops**, storing 32-bit offsets instead of 64-bit addresses to halve reference size and improve cache behaviour. This is also why heap sizing has a cliff: raising `-Xmx` from 31 GB to 33 GB can *reduce* the number of objects you can hold, because compressed oops switch off and every reference doubles in width. It is a great piece of trivia to have ready, because it sounds like a paradox and has a crisp explanation.

Where this is going. The primitive/reference split is a known wart — it is why we have both `int` and `Integer`, why generics cannot hold primitives, and why `IntStream` exists separately from `Stream<Integer>`. **Project Valhalla** is the long-running OpenJDK effort to heal it with value classes that behave like objects but are laid out flat like primitives ("codes like a class, works like an int"). It is not in a release yet. Mentioning it signals that you follow the platform, as long as you are clear it is not something you can use today.

Common mistakes

- **Using `==` on wrappers or strings.** It compares identity. Use `equals()` — and let the `Integer` cache stop surprising you by never relying on `==` in the first place.
- **Thinking `String` is a primitive.** It is a class. It feels primitive because of literals and immutability, but it is a reference type and can be `null`.
- **Expecting locals to default to zero.** Fields do; locals must be definitely assigned or it will not compile.
- **Unboxing a `null` wrapper.** `Integer x = map.get("missing"); int y = x;` throws `NullPointerException` — the unboxing call happens on `null`. Extremely common in production.
- **Overflowing an `int` before widening.** Write the `L` suffix on the first operand, not on the variable's type.
- **Using `float` / `double` for money.** Binary floating point cannot represent 0.1 exactly; `0.1 + 0.2 ≠ 0.3`. Use `BigDecimal`, constructed from a `String`.
- **Boxing in hot numeric loops.** `Long sum = 0L;` inside a million-iteration loop allocates a million `Long` objects. Declare it `long`.

Q: "What is the difference between a primitive and a reference type?"

A: A primitive variable holds the value itself in its own slot; there are exactly eight, they cannot be `null`, and `==` compares their values. A reference variable holds a handle to an object that lives on the heap; it can be `null`, and `==` compares identity rather than contents, which is why value comparison needs `equals()`. Everything that is not one of the eight primitives is a reference type.

Q: "How many primitive types are there, and what are their sizes?"

A: Eight. `byte` 8 bits, `short` 16, `int` 32, `long` 64, `float` 32, `double` 64, `char` 16 — and `boolean`, whose size the JVM spec deliberately leaves unspecified. Worth adding: `char` is the only unsigned type in Java.

Q: "Why does Java have primitives when everything else is an object?"

A: Performance and memory. A million `int`s is one contiguous 4 MB array; a million `Integer`s is a million pointers plus a million heap objects with headers — several times the memory, worse cache locality, and far more GC pressure. Project Valhalla is the long-term effort to give us object-like value types with primitive-like layout, but it has not shipped.

Q: "Is `String` a primitive?"

A: No. It is a final class in `java.lang`. It has literal syntax and is immutable, which makes it *feel* primitive, but it is a reference type — it can be `null`, and comparing two strings with `==` compares identity, not characters.

Q: "What is autoboxing, and where does it bite?"

A: Automatic conversion between a primitive and its wrapper — `Integer x = 5;` compiles to `Integer.valueOf(5)`. Two places it bites. First, unboxing a `null` wrapper throws `NullPointerException` at a line that looks like a plain

`==` behave inconsistently on top of it.

Q: "What do uninitialised variables contain?"
A: Depends where they live. Fields get default values automatically — `0`, `false`, `null`. Local variables get nothing: the compiler enforces definite assignment, so reading one before assigning is a compile error. Java never gives you the uninitialised-garbage behaviour C does.

Q: "Why is `0.1 + 0.2 ≠ 0.3`?"
A: `float` and `double` are binary IEEE 754. One tenth has no exact finite binary representation, exactly as one third has none in decimal, so the stored value is slightly off and the errors accumulate. For anything where exactness is required — money above all — use `BigDecimal`, and construct it from a `String`, because `new BigDecimal(0.1)` just captures the same inexact double.

Cheat-card

- **8 primitives**: `byte short int long float double char boolean`. Everything else is a reference type.
- Primitive slot = **the value**. Reference slot = **a handle to a heap object**.
- Primitives are **never null**; references can be.
- `==` on references = **identity**. Use `equals()` for value.
- **Fields** default to 0/false/null. **Locals** must be definitely assigned — else compile error.
- `char` is **unsigned**, 16-bit UTF-16. `boolean` size is **unspecified**.
- Generics need reference types → `List<Integer>`, never `List<int>`.
- Unboxing `null` → **NPE**. Boxing in loops → allocation storm.
- `1000L * 60 * 60 * 24 * 30` — put the `L` on the **operand**, not just the variable.
- Money → `BigDecimal` **from a String**, never `double`.

QUICK CHECK — no marks, just for you.

Java Interview Kit

1. Which of these is TRUE of Java's primitive types?

- ◯ There are exactly eight, none can be null, and char is the only unsigned one
- ◯ There are nine of them, including String
- ◯ They can be used directly as generic type arguments, as in List<int>
- ◯ They can hold null when explicitly initialised to it

Check answer

2. int[] x = {1, 2, 3}; int[] y = x; y[0] = 99; — what is x[0] afterwards, and why?

- ◯ 99, because arrays in Java are always passed by reference rather than by value
- ◯ 99, because y received a copy of the reference and both variables point at the same array object
- ◯ A compile error — arrays cannot be assigned to one another
- ◯ 1, because assigning y = x copies the array

Check answer

3. Why does `long ms = 1000 * 60 * 60 * 24 * 30;` produce a negative number?

- ◯ The literals are parsed as float and lose precision
- ◯ long cannot represent values above two billion
- ◯ The right-hand side is evaluated as int arithmetic and overflows before it is widened to long

Check answer

Previous
How Java Runs: Compilation, Bytecode and the JIT

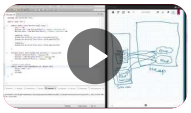
Next
Stack vs Heap: How Java Organises Memory

LEARN MORE

ARTICLES & DOCS

- Primitive data types — The Java Tutorials
  docs.oracle.com
- Types, values and variables (JLS §4)
  docs.oracle.com
- Project Valhalla — value types for the JVM
  openjdk.org

VIDEOS



Java Pass By Value Stack Heap Memory Explanation
Diyansh IT Services

Hand-picked references for this lesson.